



InfiniFi

Security Review



Disclaimer

Security Review

InfiniFi



Disclaimer

The ensuing audit offers no assertions or assurances about the code's security. It cannot be deemed an adequate judgment of the contract's correctness on its own. The authors of this audit present it solely as an informational exercise, reporting the thorough research involved in the secure development of the intended contracts, and make no material claims or guarantees regarding the contract's post-deployment operation. The authors of this report disclaim all liability for all kinds of potential consequences of the contract's deployment or use. Due to the possibility of human error occurring during the code's manual review process, we advise the client team to commission several independent audits in addition to a public bug bounty program.

Table of Contents

Security Review

InfiniFi



Table of Contents

Summary Security Review

InfiniFi



Summary

Three Sigma audited InfiniFi in a 2 person week engagement. The audit was conducted from 13/07/2026 to 17/07/2026.

Protocol Description

InfiniFi is a DeFi protocol that enables users to mint and redeem receipt tokens (iUSD, iETH) against collateral assets (USDC, ETH). The protocol features a sophisticated yield generation system through multiple farm integrations, a locking mechanism for enhanced rewards, and a governance system for farm allocation voting.

Scope Security Review

InfiniFi



Scope

Filepath	nSLOC
src/integrations/outland/portals/PortalHub.sol	206
src/integrations/outland/portals/PortalBase.sol	203
src/gateway/OutlandGatewayV1.sol	191
src/integrations/outland/OutlandVault.sol	187
src/integrations/outland/portals/PortalOutpost.sol	149
src/periphery/SIUSDC_L2.sol	64
src/integrations/outland/OutlandMsgCodec.sol	49
src/tokens/InfiniFiL2Token.sol	38
src/gateway/InfiniFiGatewayV4.sol	20
src/finance/AccountingL2.sol	19
Total	1126

Methodology Security Review

InfiniFi



To begin, we reasoned meticulously about the contract's business logic, checking security-critical features to ensure that there were no gaps in the business logic and/or inconsistencies between the aforementioned logic and the implementation. Second, we thoroughly examined the code for known security flaws and attack vectors. Finally, we discussed the most catastrophic situations with the team and reasoned backwards to ensure they are not reachable in any unintentional form.

Taxonomy

In this audit, we classify findings based on ImmuneFi's [Vulnerability Severity Classification System \(v2.3\)](#) as a guideline. The final classification considers both the potential impact of an issue, as defined in the referenced system, and its likelihood of being exploited. The following table summarizes the general expected classification according to impact and likelihood; however, each issue will be evaluated on a case-by-case basis and may not strictly follow it.

Impact / Likelihood	LOW	MEDIUM	HIGH
NONE	None		
LOW	Low		
MEDIUM	Low	Medium	Medium
HIGH	Medium	High	High
CRITICAL	High	Critical	Critical

Project Dashboard Security Review

InfiniFi



Project Dashboard

Application Summary

Name	InfiniFi
Repository	https://github.com/InfiniFi-Labs/infinifi-contracts
Commit	PR395:8c61ae6
Language	Solidity
Platform	Ethereum

Engagement Summary

Timeline	13/07/2026 to 17/07/2026
Nº of Auditors	2
Review Time	2 person weeks

Vulnerability Summary

Issue Classification	Found	Addressed	Acknowledged
Critical	0	0	0
High	0	0	0
Medium	2	2	0
Low	2	0	2
None	0	0	0

Category Breakdown

Suggestion	0
Documentation	0
Bug	4
Optimization	0
Good Code Practices	0

Risk Section Security Review

InfiniFi



Risk Section

No risks were identified.

Findings

Security Review

InfiniFi



Findings

3S-M01

Nonzero OutlandGatewayV1::unstake fees can block SIUSDC_L2 withdrawals when idle liquidity is exhausted

Id	3S--M01
Classification	Medium
Impact	Medium
Likelihood	Medium
Category	Bug
Status	Addressed in #7d2fc4e .

Description

SIUSDC_L2 is an ERC-4626 vault that invests deposited USDC into siUSD. When its idle USDC balance is insufficient, **SIUSDC_L2::_withdraw** first calls **OutlandGatewayV1::unstake**, then redeems the returned iUSD for the remaining USDC amount.

However, **OutlandGatewayV1::unstake** deducts the configured fee:

```
uint256 iusdOut = _amountIn.mulWadDown(iusdPerSiusd);
iusdOut = _applyFee(iusdOut);
```

SIUSDC_L2::_withdraw still passes the fee-free amount as **minAssetsOut**:

```
uint256 iusdAmount = gateway.unstake(address(this), siUSDAmount);
gateway.redeem(address(this), iusdAmount, assets - idle);
```

With a 5 bps fee and zero idle USDC, a withdrawal expecting 100 USDC may receive only 99.95 USDC. **OutlandGatewayV1::redeem** then reverts because the output is below the required minimum. Consequently, every withdrawal that requires siUSD unwinding fails until liquidity is added, the fee is disabled, or the contracts are fixed. The transaction reverts atomically, so no partial withdrawal occurs.

The client indicated that fees are intended for minting/staking protection and should not be charged during unstaking.

Steps to Reproduce

1. Configure a nonzero gateway fee, such as 5 bps.
2. Deposit USDC into **SIUSDC_L2** until the USDC idle balance is zero and the funds are converted to siUSD.
3. Request a withdrawal requiring siUSD unwinding.
4. Observe that **unstake** returns a fee-reduced iUSD amount.
5. Observe that **redeem** reverts because the received USDC is below the fee-free **minAssetsOut**.

Recommendation

Do not charge the gateway fee during **OutlandGatewayV1::unstake**, keep the fee only on the intended minting/staking paths.

```
uint256 iusdOut = _amountIn.mulWadDown(iusdPerSiusd);
- iusdOut = _applyFee(iusdOut);
```

Add regression tests covering nonzero fees, zero idle liquidity, and partial idle liquidity. If an unstake fee is retained instead, all withdrawal calculations and ERC-4626 previews must use the post-fee amount.

3S-M02

Shared asset mapping in PortalHub::bridge allows mapped USDC to poison the outpost nonce queue & lock user funds

Id	3S--M02
Classification	Medium
Impact	Medium
Likelihood	Medium
Category	Bug
Status	Addressed in #f9bae0d .

Description

Although the bridge flow is intended for iUSD and siUSD, **InfiniFiGatewayV4::bridge()** forwards any token and **PortalHub::_handleSendTokenBridge()** accepts any token present in the generic **assetMapping**:

- **PortalHub.sol:193-198** validates only the mapping and transfers the token into the destination **OutlandVault**.
- **PortalHub.sol:201-213** then queues a **TOKEN_BRIDGE** message containing the mapped destination token.
- **PortalOutpost.sol:227-230** accepts only the configured L2 receipt or staked token. A mapped L2 USDC address therefore causes processing to revert with **InvalidBridgedToken**.

Because **PortalBase::processMessage()** executes the handler before deleting the queued hash and advancing the nonce (**PortalBase.sol:127-138**), the invalid USDC message remains at the expected nonce and blocks all later messages from that hub/outpost queue, including legitimate bridges, rate updates, and reserve transfers.

The user's USDC is already held by the destination vault before the asynchronous message fails. **setInboundNonce()** can skip the failed message, but it does not move those tokens back. **OutlandVault::portalBridgeWithdraw()** only supports the canonical receipt and staked tokens (**OutlandVault.sol:220-231**), and there is no automatic user-refund path for the deposited USDC. The funds are therefore effectively locked under the current implementation unless governance performs a separate rescue, migration, or contract upgrade.

The attacker does not need administrative access, USDC is legitimately mapped for reserve transfers, so an ordinary user can submit a minimal mapped-USDC bridge & poison the queue. The attacker's own USDC is sacrificed and is not a profit source. The impact is queue disruption and potential loss of access to funds for affected bridge users.

scope: [InfiniFi-Labs/infinifi-contracts#395](https://github.com/InfiniFi-Labs/infinifi-contracts#395)

Risk:

> The user's USDC is already held by the destination vault before the asynchronous message fails. `setInboundNonce()` can skip the failed message, but it does not move those tokens back. `OutlandVault::portalBridgeWithdraw()` only supports the canonical receipt and staked tokens (`OutlandVault.sol:220-231`), and there is no automatic user-refund path for the deposited USDC. The funds are therefore effectively locked under the current implementation unless governance performs a separate rescue, migration, or contract upgrade.

Recommendation

Restrict `_handleSendTokenBridge()` to the destination vault's canonical receipt or staked token before transferring funds:

```
OutlandVault vault = OutlandVault(vaults.get(_chainId));
require(
    _token == vault.receiptToken() || _token == vault.stakedToken(),
    InvalidAsset(_token)
);
require(assetMapping[_chainId].contains(_token), InvalidAsset(_token));
```

Keep the generic asset mapping for `sendTokens()`, since reserve assets such as USDC still need that separate transfer flow.

Additionally, add an explicit cancellation/refund mechanism for already-queued unsupported-token bridges. It should return the deposited asset to the original sender and should not rely on nonce skipping alone.

3S-M03

Receipt-first supply reconciliation causes valid L2 reports to block the outpost message lane

Id	3S--M03
Classification	Medium
Impact	Medium
Likelihood	Medium
Category	Bug
Status	Addressed in #e24aaab .

Description

OutlandVault::_syncL2Supply mirrors an L2's iUSD and siUSD supplies on L1. A normal unstake on L2 reduces siUSD and increases loose iUSD without creating new value; the same position has only changed form.

The issue is in **src/integrations/outland/OutlandVault.sol:283-318**. The function reconciles the new iUSD balance before removing the old siUSD balance:

```

ReceiptToken(receiptToken).mint(address(this), receiptTokensToMint);
// Executed later during siUSD reconciliation
InfiniFiGateway(gateway).unstake(address(this), stakedTokensToBurn);

```

Assume the vault currently mirrors **100 siUSD** and no loose iUSD. The next valid L2 report contains **100 iUSD** and no siUSD, with total assets unchanged. **_syncL2Supply** first mints **100 iUSD** while the old **100 siUSD** position—and its underlying iUSD—still exists. The same backing is therefore counted against both positions, temporarily increasing **ReceiptToken::totalSupply** without increasing protocol assets. Under the calculation in **src/finance/YieldSharing.sol:132-141**, **YieldSharing::unaccruedYield** becomes negative.

The function then calls **InfiniFiGatewayV2::unstake** in **src/gateway/InfiniFiGatewayV2.sol:136-148**, which attempts to redeem the excess siUSD. However, **StakedToken::maxRedeem** explicitly disables redemptions while unaccrued yield is negative:

```

if (paused() || YieldSharing(yieldSharing).unaccruedYield() < 0) {
    return 0;
}

```

```
}
```

The redemption consequently reverts with **ERC4626ExceededMaxRedeem**. Although the revert rolls back the temporary mint, it also prevents the valid L2 state from being applied.

The impact extends to cross-chain message processing. In **src/integrations/outland/portals/PortalBase.sol:122-140**, **PortalBase::processMessage** executes the report before deleting it and advancing the inbound nonce.

Because execution reverts, the report remains at the expected nonce and every later message from that outpost is blocked. This includes later accounting reports and token transfers. If tokens were already burned for a later bridge, the destination transfer remains pending until the lane and mirrored supply state are repaired.

PoC

paste the following PoC in **test/unit/outland/OutlandVault.t.sol** :

```
function testPoCReceiptFirstSyncRevertsSiusdToIusdTransition() public {
    // In this fixture iUSD is priced at $0.50, so $55 of L2 assets represents
    // 110 iUSD of backing. This leaves positive yield over the 100-token mirror.
    vm.prank(portal);
    vault.portalUpdate(55e18, 0, 100e18);
    uint256 mirroredSiusdBefore = siusd.balanceOf(address(vault));
    uint256 lastPortalUpdateBefore = vault.lastPortalUpdate();
    assertEq(iusd.balanceOf(address(vault)), 0);
    assertApproxEqAbs(mirroredSiusdBefore, 100e18, 1);
    assertGt(yieldSharing.unaccruedYield(), 0);
    vm.warp(block.timestamp + 1);
    // L2 users have now unstaked the 100 siUSD into 100 loose iUSD while
    // total L2 assets remain unchanged. _syncL2Supply mints the loose iUSD
    // first, temporarily making unaccruedYield negative. The subsequent
    // siUSD redemption sees maxRedeem() == 0 and reverts.

    vm.expectPartialRevert(bytes4(keccak256("ERC4626ExceededMaxRedeem(address,uint256,uint256)")));
    vm.prank(portal);
    vault.portalUpdate(55e18, 100e18, 0);
    // The complete portal update rolled back, leaving the old mirror intact.
    assertEq(iusd.balanceOf(address(vault)), 0);
    assertEq(siusd.balanceOf(address(vault)), mirroredSiusdBefore);
    assertEq(vault.lastPortalUpdate(), lastPortalUpdateBefore);
}
```

```
}
```

Logs:

```
Ran 1 test for test/unit/outland/OutlandVault.t.sol:OutlandVaultUnitTest
[PASS] testPoCReceiptFirstSyncRevertsSiusdTolusdTransition() (gas: 752318)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 16.05ms (1.76ms CPU time)
```

Steps to Reproduce

1. Call **OutlandVault::portalUpdate** with unchanged backing assets, **0** loose iUSD, and **100** siUSD.
2. Submit the next valid report with the same assets, **100** iUSD, and **0** siUSD.
3. Observe that **OutlandVault::_syncL2Supply** mints the new iUSD before removing the old siUSD, making **YieldSharing::unaccruedYield** negative.
4. Observe that **InfiniFiGatewayV2::unstake** reverts with **ERC4626ExceededMaxRedeem** because **StakedToken::maxRedeem** returns zero.
5. Execute the same report through **PortalBase::processMessage** and confirm that the inbound nonce does not advance and the next message from that outpost cannot be processed.

Recommendation

Reconcile supply reductions before supply increases. In **OutlandVault::_syncL2Supply**, remove excess siUSD and burn the iUSD returned by **unstake** before minting any increase in loose iUSD.

```
uint256 currentReceiptTokens = ReceiptToken(receiptToken).balanceOf(address(this));
+ uint256 receiptTokensToMint;
....
else if (currentReceiptTokens < _receiptTokenTotalSupply) {
-   uint256 receiptTokensToMint = _receiptTokenTotalSupply - currentReceiptTokens;
-   ReceiptToken(receiptToken).mint(address(this), receiptTokensToMint);
+   receiptTokensToMint = _receiptTokenTotalSupply - currentReceiptTokens;
}
....
else if (currentStakedTokens < _stakedTokenTotalSupply) {
    // existing staking logic
```

```
    }  
    ....  
+    // Mint new loose iUSD after the siUSD reconciliation.  
+    if (receiptTokensToMint > 0) {  
+        ReceiptToken(receiptToken).mint(address(this), receiptTokensToMint);  
+    }
```

3S-M04

Re-timestamping delayed rate messages as fresh enables stale-rate arbitrage and under-backed iUSD claims

Id	3S--M04
Classification	Medium
Impact	Medium
Likelihood	Medium
Category	Bug
Status	Addressed in #72bd6d3 .

Description

PortalHub::sendKeyValue forwards the **IUSD_RATE** and **IUSD_SIUSD_RATE** values to an L2 outpost in [PR #395](#). However, the payload contains no source timestamp.

PortalBase::receiveMessage only stores the message hash. When a keeper later processes the message, **PortalOutpost::_handleReceiveKeyValue** records the processing timestamp:

```
keyValues[_key] = _value;
keyValuesUpdatedAt[_key] = block.timestamp;
```

OutlandGatewayV1::_getKeyUint256 then validates freshness against that processing timestamp:

```
require(block.timestamp < updatedAt + validityDuration, StaleKeyValue(_key));
```

Therefore, a rate sent at time **T** and processed after delay **d** remains valid until **T + d + validityDuration**, instead of expiring at **T + validityDuration**. During the review, the supplied Arbitrum trace showed approximately 67 minutes between rate delivery and processing while the configured validity window was one hour, demonstrating that this extension can occur in practice.

If the delayed message contains an old **IUSD_SIUSD_RATE**, a user can stake iUSD using the old rate and later unstake using a newer favorable rate. With the current fee applied to both paths, the round trip approximately returns:

$$\text{input iUSD} \times (\text{newRate} / \text{oldRate}) \times (1 - \text{fee})^2$$

If the newer rate reflects yield earned before the user entered, or is not yet backed by L2 assets, the user receives extra iUSD that can be redeemed from shared farm liquidity. Existing holders then bear the shortfall, while the protocol's supply and backing become inconsistent. A favorable rate increase that represents genuine yield earned after the user entered is not itself an exploit, the issue is the acceptance of a message that should already be stale.

Steps to Reproduce

1. Configure an outpost with a one-hour **validityDuration** and an initial **IUSD_SIUSD_RATE**.
2. Send the rate from the Hub through **PortalHub::sendKeyValue**.
3. Delay connector delivery or keeper processing beyond the validity window.
4. Queue the message through **PortalBase::receiveMessage**, then process it through **PortalBase::processMessage**.
5. Observe that **keyValuesUpdatedAt** is set to the processing timestamp and the expired rate is accepted by **OutlandGatewayV1::stake** or **unstake**.
6. Process a newer favorable rate. Stake using the old rate, then unstake using the newer rate and compare the resulting iUSD balance with the original amount.
7. If the rate increase predates the user's entry or lacks corresponding backing, redeem the excess iUSD and observe the resulting shortfall in shared liquidity.

Recommendation

Include an authenticated source timestamp or effective epoch in every key-value payload. Set it when **PortalHub::sendKeyValue** is called, preserve it in **PortalOutpost**, and use it in **OutlandGatewayV1::_getKeyUint256** instead of **block.timestamp**.

The paired **IUSD_RATE** and **IUSD_SIUSD_RATE** values should share an epoch or update identifier so users cannot combine values from different rate updates. Expired messages also need an explicit cancellation or skip mechanism, simply reverting an expired head message would leave the inbound nonce stuck and block newer valid updates.

3S-L01

Hub relays and refunds depend on prefunded connector reserves, risking message-processing delays

Id	3S--L01
Classification	Low
Impact	Medium
Likelihood	Low
Category	Bug
Status	Acknowledged

Description

OutlandGatewayV1::bridge routes L2-to-L2 transfers through the hub. The user's **msg.value** pays only for the first outpost-to-hub message, while the hub-to-destination message is sent by **PortalHub::_handleSendTokenBridge** with no attached value:

```
IOutlandConnector(_connector).sendMessage{
    value: _pullFromSender ? msg.value : 0
}(_chainId, abi.encode(payload));
```

The same behavior applies when an unsupported destination requires a refund message to the source outpost. Because **PortalBase::processMessage** is non-payable, the keeper cannot provide the required fee while processing either message. Instead, **ConnectorLZ::sendMessage** and **ConnectorCCIP::sendMessage** pay the freshly quoted fee from their existing native balance and revert when that balance is insufficient.

If this occurs, the user's tokens have already been burned on the source outpost, leaving the bridge pending until the hub connector is funded. Since inbound messages must be processed sequentially, the failed message also blocks later messages from the same source chain. Operators can recover by funding the connector and retrying the message; alternatively, advancing the inbound nonce would skip rather than complete it.

As per discussion with team, they acknowledged this issue, but it requires the protocol to continuously monitor and replenish connector balances. Otherwise, relays and refunds may remain pending. It also makes the protocol responsible for paying all second-hop and refund fees (fee for refund back to outpost).

Steps to Reproduce

1. Leave the hub connector with insufficient native currency for a hub-to-outpost message.
2. Have an allowed user call **OutlandGatewayV1::bridge** for an L2-to-L2 transfer while paying the first-hop fee.
3. After the first message succeeds and the user's tokens are burned, call **PortalBase::processMessage** on the hub.
4. Observe that the second-hop call forwards zero value and reverts due to the connector's insufficient balance.
5. Observe that the bridge remains pending and the inbound nonce does not advance until the connector is funded.

Recommendation

If the current fee design is retained, document it and implement minimum-balance monitoring and automated replenishment for hub connectors.

If users should pay both fees, require a signed quote covering both bridge legs and verify it in the gateway. Since the second fee is collected on the source chain but spent on the hub, this approach must be paired with an off-chain settlement mechanism that funds the corresponding hub connector.

3S-L02

Cached-rate arbitrage during valid rate transitions

Id	3S--L02
Classification	Low
Impact	Medium
Likelihood	Low
Category	Bug
Status	Acknowledged

Description

OutlandGatewayV1 allows users to convert using a cached rate for the full **validityDuration**. A user may enter using the previous rate and exit after a favorable **IUSD_SIUSD_RATE** update is processed.

e.g a user may stake iUSD at the old siUSD rate, wait for the rate to increase, and then unstake at the newer rate. The staking fee reduces the gain, but does not eliminate it if the rate increase exceeds the fee.

The direct iUSD path is less concerning because **IUSD_RATE** is expected to remain close to \$1.

A user may capture a portion of yield that accrued before their entry, with the difference effectively coming from shared farm liquidity and existing holders. The impact is bounded by the rate change and available liquidity.

Recommendation

Consider applying a time-based or rate-transition fee, delaying use of newly updated rates, or binding conversions to an effective rate epoch. Otherwise, document that users may benefit from rate changes during the validity window.

3S-L03

Inconsistent fee denomination leads to unequal siUSD output

Id	3S--L03
Classification	Low
Impact	Low
Likelihood	Low
Category	Suggestion
Status	Addressed in #9dbc038 .

Description

OutlandGatewayV1::mintAndStake applies the fee to the iUSD amount before converting it to siUSD, while **OutlandGatewayV1::stake** first converts iUSD to siUSD and then applies the fee. Since both **_applyFee** and the iUSD-to-siUSD conversion round down, the two paths can mint different amounts for the same iUSD value.

For example, with **100e18** iUSD, an iUSD/siUSD rate of **1.03e18**, and a **1%** fee, **mintAndStake** mints **96116504854368932038** siUSD, while **stake** mints **96116504854368932039** siUSD. The **stake** path therefore charges one siUSD wei less. Although the difference is only rounding dust, fee handling is inconsistent across equivalent staking flows.

Steps to Reproduce

1. Set the iUSD/siUSD rate to **1.03e18** and the gateway fee to **0.01e18**.
2. Calculate the siUSD minted by **OutlandGatewayV1::mintAndStake** for an iUSD value of **100e18**; the result is **96116504854368932038**.
3. Call **OutlandGatewayV1::stake** with **100e18** iUSD; the result is **96116504854368932039**.
4. Observe that **stake** mints one additional siUSD wei.

Recommendation

Apply the fee to the iUSD input before converting it to siUSD in **OutlandGatewayV1::stake**. This keeps the fee calculation consistently denominated in iUSD across all staking and unstaking flows.

- uint256 siusdOut = _amountIn.divWadDown(iusdPerSiusd);
- siusdOut = _applyFee(siusdOut);
- + uint256 iusdAfterFee = _applyFee(_amountIn);
- + uint256 siusdOut = iusdAfterFee.divWadDown(iusdPerSiusd);

3S-L03

Allowing a 100% fee can reduce user output to zero

Id	3S--L03
Classification	Low
Impact	Low
Likelihood	Low
Category	Bug
Status	Addressed in #a6e5dd9 .

Description

OutlandGatewayV1::setFee allows the **PROTOCOL_PARAMETERS** role to set the fee to **1e18**, or 100%. At this value, **_applyFee** reduces the output of **mintAndStake**, **stake**, and **unstake** to zero while still consuming the user's input.

Although the function is role-restricted, an incorrect fee configuration could cause users to receive nothing. These functions do not accept a minimum output, so users cannot protect themselves from an excessive fee.

Steps to Reproduce

1. Call **OutlandGatewayV1::setFee** with **1e18** using the **PROTOCOL_PARAMETERS** role.
2. Call **OutlandGatewayV1::stake** with a nonzero iUSD amount.
3. The iUSD is burned, but the user receives zero siUSD.

Recommendation

Set a conservative maximum fee below 100% and add minimum-output checks to **mintAndStake**, **stake**, and **unstake**.